



FLASH: Provenance Graph Representation Learning for Intrusion Detection

IEEE Symposium on Security and Privacy

YUG SHETH & SHREY SOJITRA

The APT Problem: Why Traditional IDS Falls Short



Advanced Persistent Threats (APTs)

APTs are long-horizon, multi-stage attacks — phishing, lateral movement, exfiltration — designed to evade signature-based detection by mimicking legitimate behavior over extended periods.

Why Traditional IDS Fails

- Signature-based systems miss novel attack variants
- Anomaly detectors lack semantic context
- No causal tracing across system events
- High false positive rates burden SOC analysts

Problem Statement: Gaps in Provenance-Based IDS

Existing provenance-based systems leave critical detection gaps across four dimensions:

Semantic Neglect

Node attributes — file paths, process names, IP addresses — are encoded as raw strings or one-hot vectors, losing semantic meaning

Temporal Blindness

Event ordering within system logs is discarded; temporal context critical for multi-stage attack detection is lost

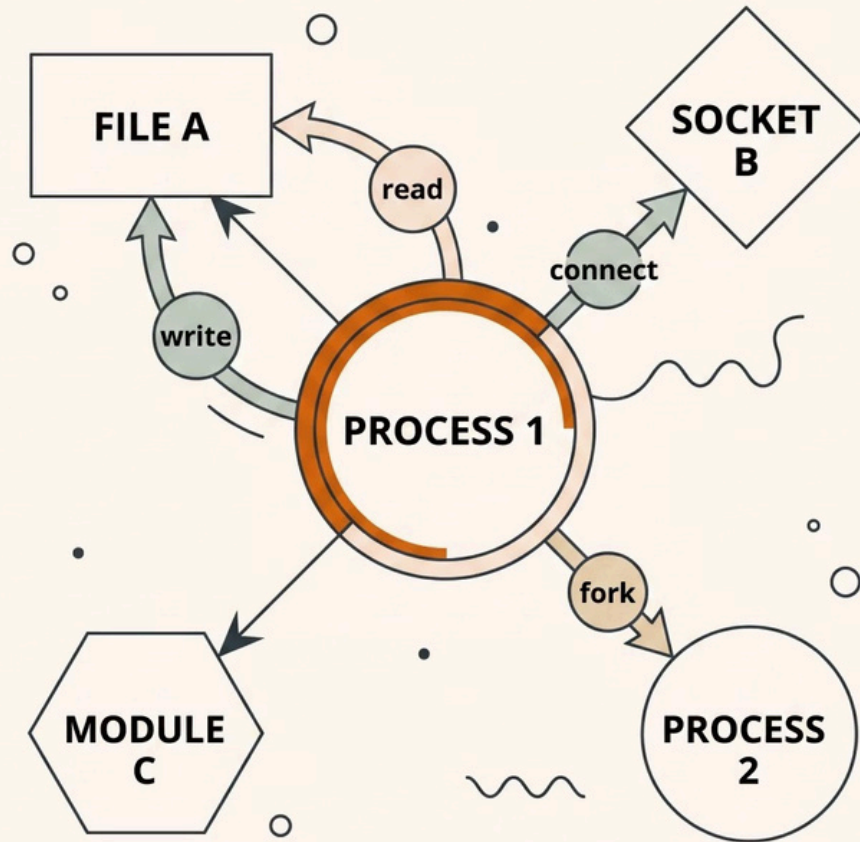
Scalability Bottleneck

Full GNN recomputation for every new event creates prohibitive runtime overhead in production environments

Coarse Detection

Graph-level anomaly scoring flags entire subgraphs rather than pinpointing malicious nodes, generating excessive false positives

Data Provenance: The Foundation



What is a Provenance Graph?

A **directed acyclic graph** that captures causal relationships between OS-level entities derived from system audit logs.

Node Types

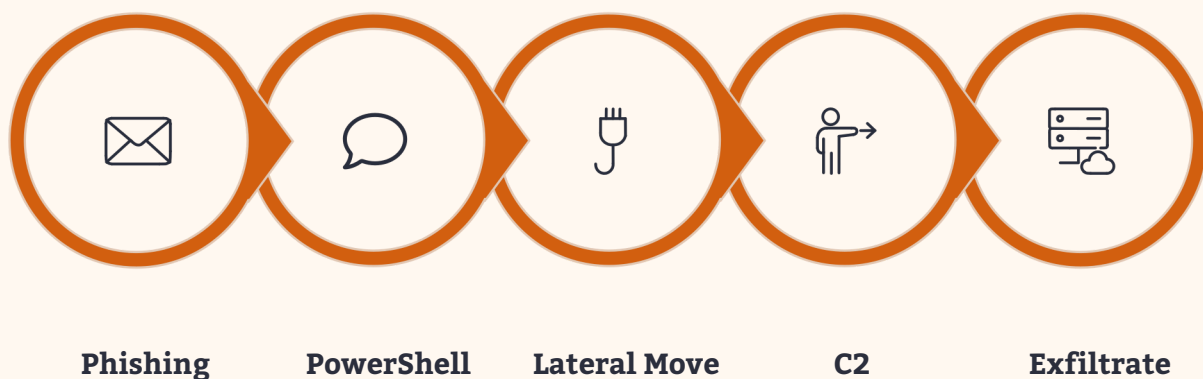
- **Process** — executing programs and their attributes
- **File** — filesystem objects with full path context
- **Socket** — network endpoints (IP:port tuples)
- **Module** — shared libraries and kernel modules

Edges

Directed system call events (read, write, fork, exec, connect)

with timestamps preserving causal ordering.

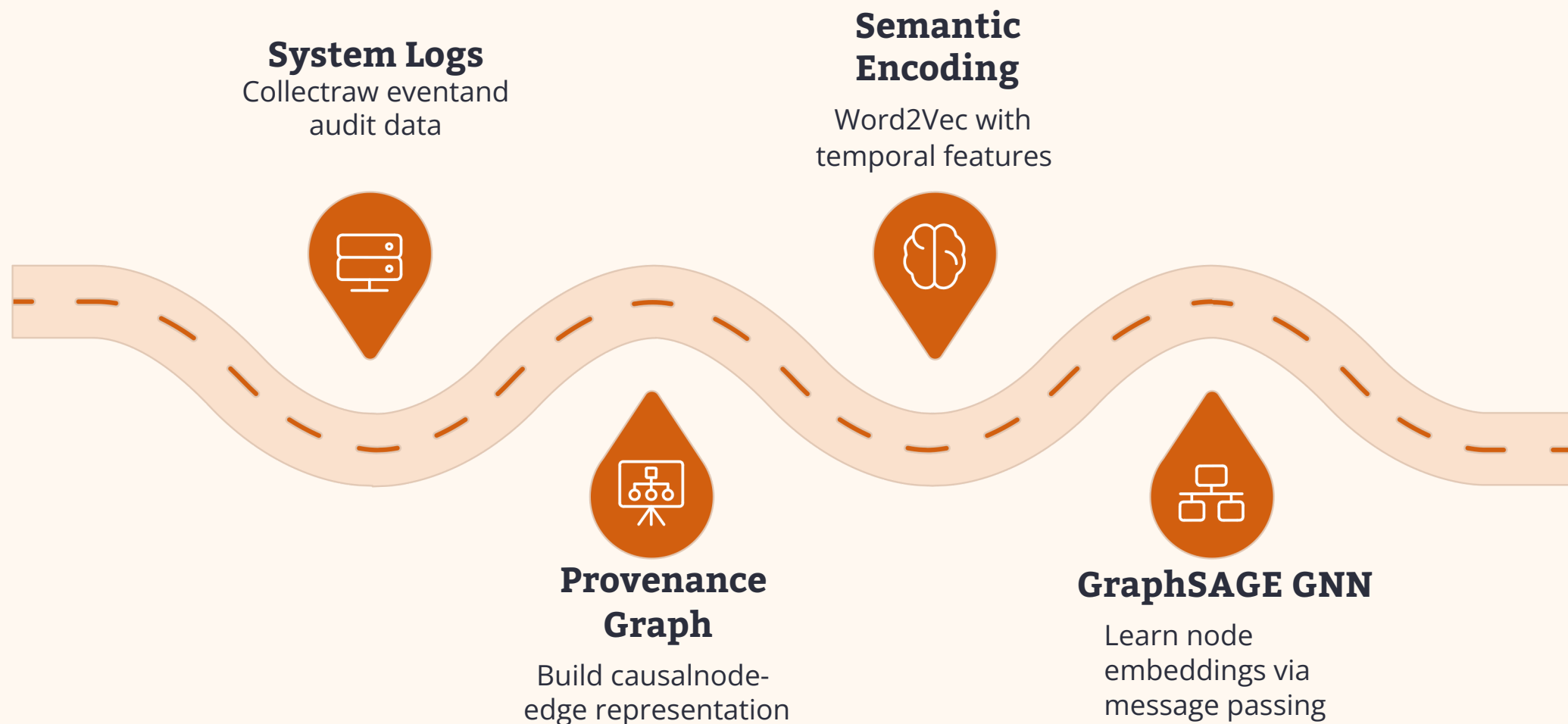
Real-World Attack Scenario: DARPA OpTC



The DARPA Operationally Transparent Cyber (OpTC) dataset captures this full kill chain across enterprise hosts — provenance graphs preserve every causal link from initial compromise to exfiltration.



FLASH Architecture: End-to-End Pipeline



FLASH integrates semantic encoding, graph representation learning, and a persistent embedding cache to deliver **real-time, node-level** intrusion detection — avoiding repeated GNN computation via the embedding recycling database.

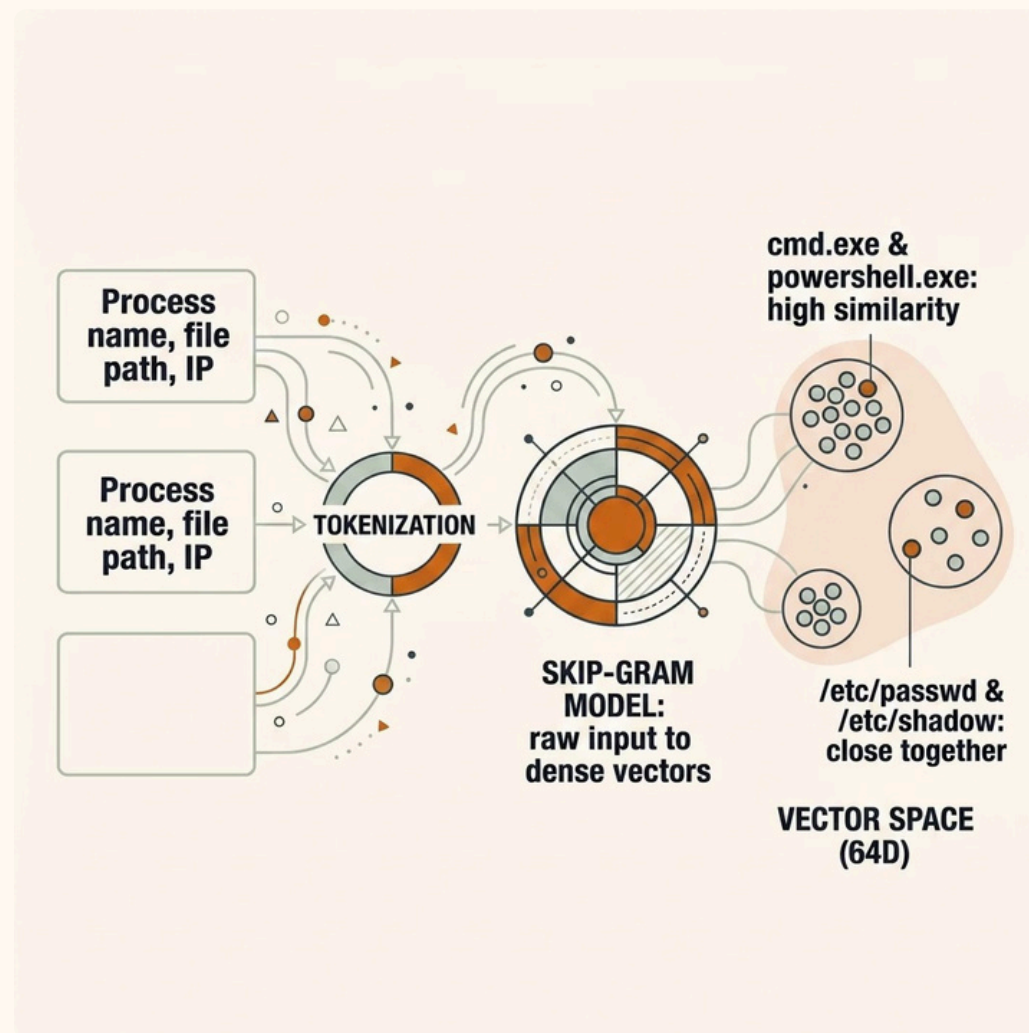
Semantic Attribute Encoding with Word2Vec

What Gets Encoded

- Process names and command-line arguments
- File paths (tokenized by delimiter)
- IP addresses and port numbers
- System call types and flags

Why Word2Vec Over One-Hot?

- Dense 64-dim vectors capture semantic proximity
- `cmd.exe` and `powershell.exe` cluster meaningfully
- Generalizes to unseen process names at inference
- Fixed-length vectors regardless of vocabulary size



Temporal Encoding: Preserving Event Order

Positional Encoding Formula

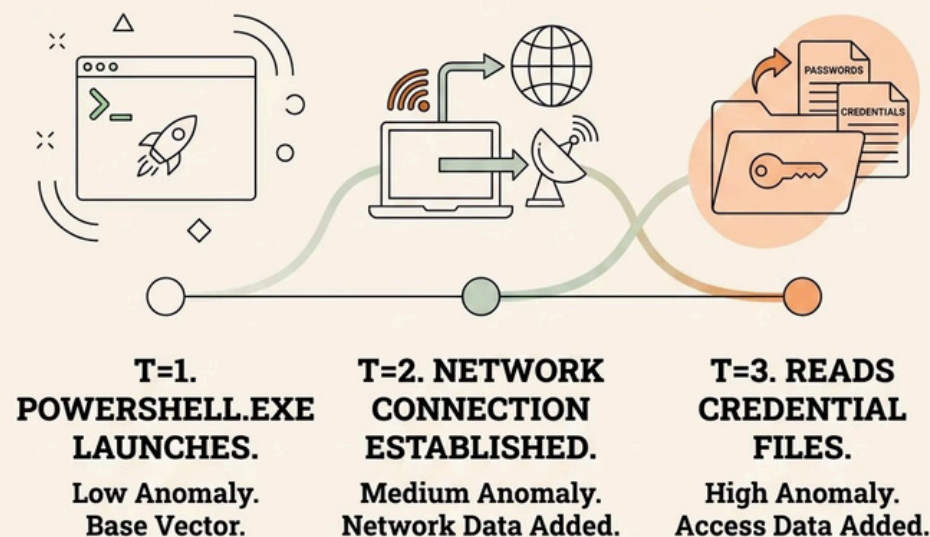
Inspired by transformer positional encoding, FLASH injects timestamp position t into node attribute vectors:

$$PE_t(, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

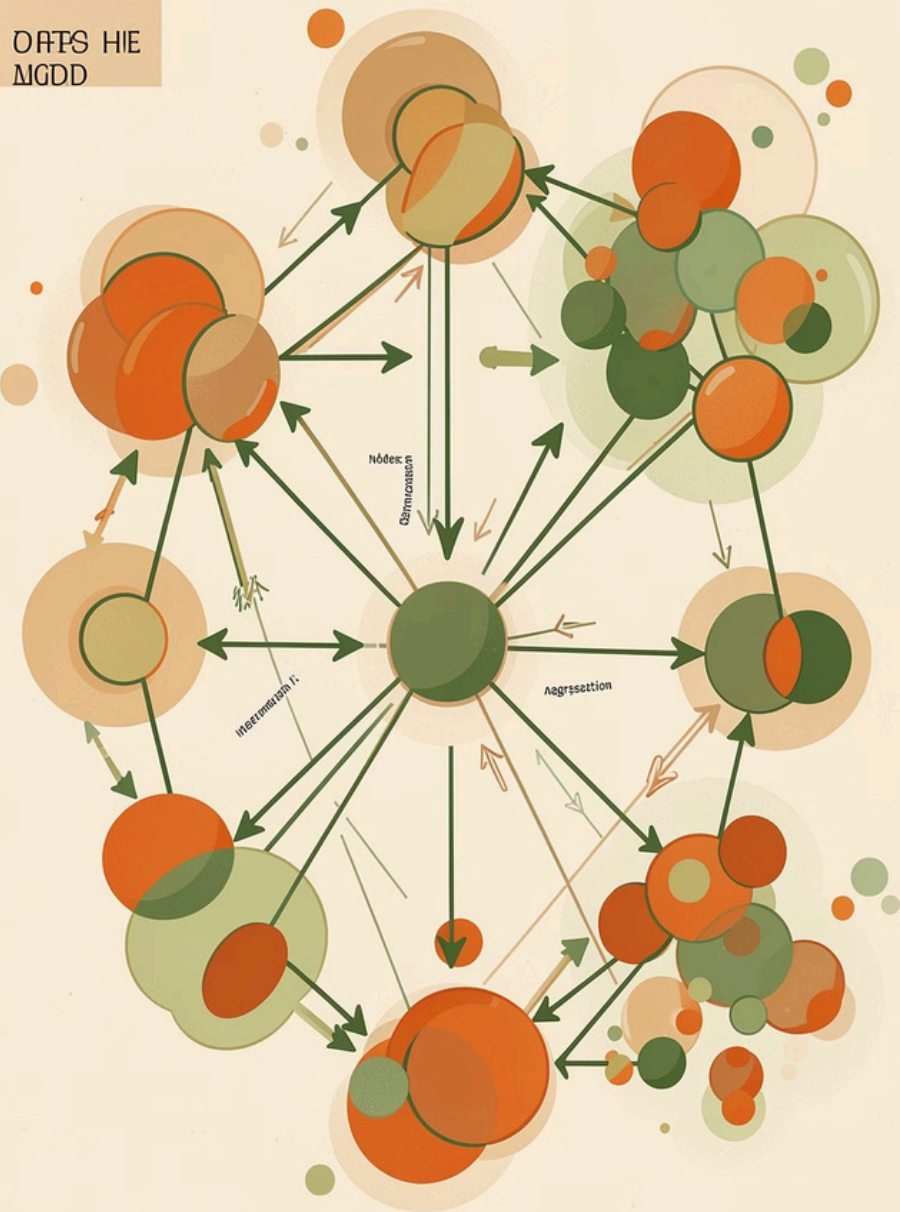
$$PE_t(, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

$$e^{final} = e_{Word2Vec} + PE(t)$$

This ensures two identical processes executing at different times produce **distinct embeddings**, enabling the GNN to reason about event sequences rather than snapshots.



TEMPORAL ENCODING MAKES EACH EVENT VIGOROUSLY UNIQUE, UNLIKE IDENTICAL STATIC VIEWS.



GraphSAGE: Graph Neural Network Core



Sample Neighborhood

For each node, uniformly sample k neighbors across 2 hops — bounded complexity regardless of graph size



Aggregate Features

Apply mean aggregation over sampled neighbor embeddings:

$$h_{N_v}^k = \text{MEAN}(\{h_u^{k-1}, \forall u \in N_v\})$$

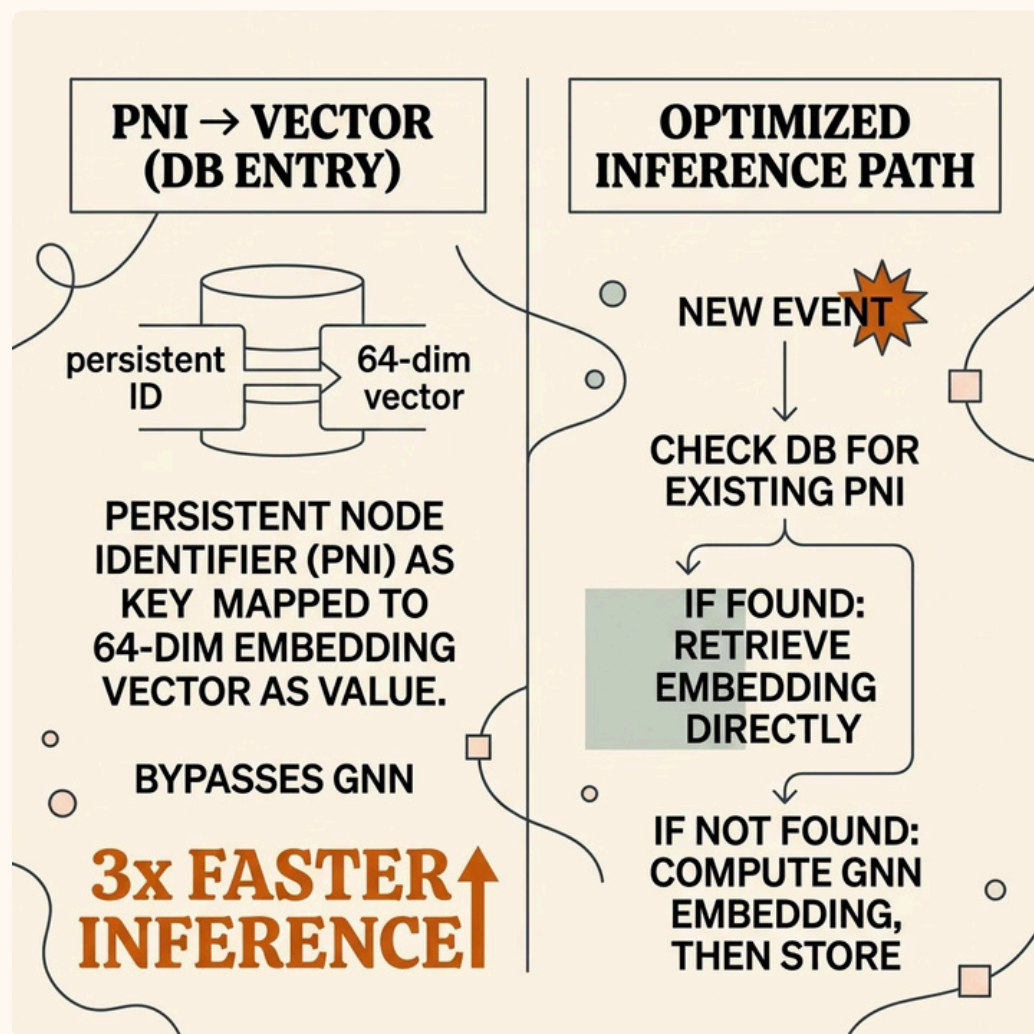


Update Node Embedding

Concatenate and apply learned weight matrix:

$$h_v^k = \sigma(W^k \cdot [h_v^{k-1} \parallel h_{N_v}^k])$$

Embedding Recycling Database



The Core Innovation

FLASH's most impactful scalability mechanism — a persistent key-value store indexed by **Persistent Node Identifiers (PNI)**.

How It Works

- PNI uniquely identifies each provenance node across sessions
- Computed embeddings are stored after first GNN pass
- Subsequent events reuse cached embeddings — no recomputation
- Only truly new or modified nodes trigger GNN inference

✓ Result: **3× faster inference** with negligible accuracy loss on DARPA OpTC benchmarks